EXPERIMENT-25

25. RSA Algorithm – Common Factor Attack

AIM

To write a C program demonstrating how knowing that an RSA plaintext block has a common factor with the modulus \(n=pq\) can help determine the private key.

PROCEDURE

1. Read the RSA modulus \(n\), public exponent \(e\), and a plaintext block \(m\).
2. Calculate the greatest common divisor (GCD) of \(m\) and \(n\).
3. If \(1 < \gcd(m,n) < n\), the GCD reveals a non-trivial factor of \(n\).
4. Calculate the other factor of \(n\).
5. Compute \(\phi(n)=(p-1)(q-1)\).
6. Use the Extended Euclidean Algorithm to calculate the private exponent \(d\).

CODE (C)

```
#include <stdio.h>

int gcd(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main(void) {
    int n, e, m, g;
    int p, q, phi;
    int a, b, x0 = 0, x1 = 1, d;

    printf("Enter RSA modulus n: ");
    scanf("%d", &n);
```

```
printf("Enter public exponent e: ");
scanf("%d", &e);

printf("Enter plaintext block m: ");
scanf("%d", &m);

g = gcd(m, n);
printf("GCD(m, n) = %d\n", g);

if (g <= 1 || g >= n) {
    printf("No non-trivial factor of n found.\n");
    printf("The plaintext block does not reveal a factor.\n");
    return 0;
}

p = g;
q = n / p;

if (p * q != n) {
    printf("Invalid factorization.\n");
    return 1;
}

phi = (p - 1) * (q - 1);

a = e;
b = phi;

while (b != 0) {
    int quotient = a / b;
    int remainder = a % b;
    int temp = x0;

    a = b;
    b = remainder;

    x0 = x1 - quotient * x0;
    x1 = temp;
}

if (a != 1) {
    printf("Private exponent does not exist.\n");
    return 1;
}

d = x1 % phi;
if (d < 0)
    d += phi;
```

```
    printf("p = %d\n", p);
    printf("q = %d\n", q);
    printf("phi(n) = %d\n", phi);
    printf("Private exponent d = %d\n", d);
    printf("Private key = (%d, %d)\n", d, n);

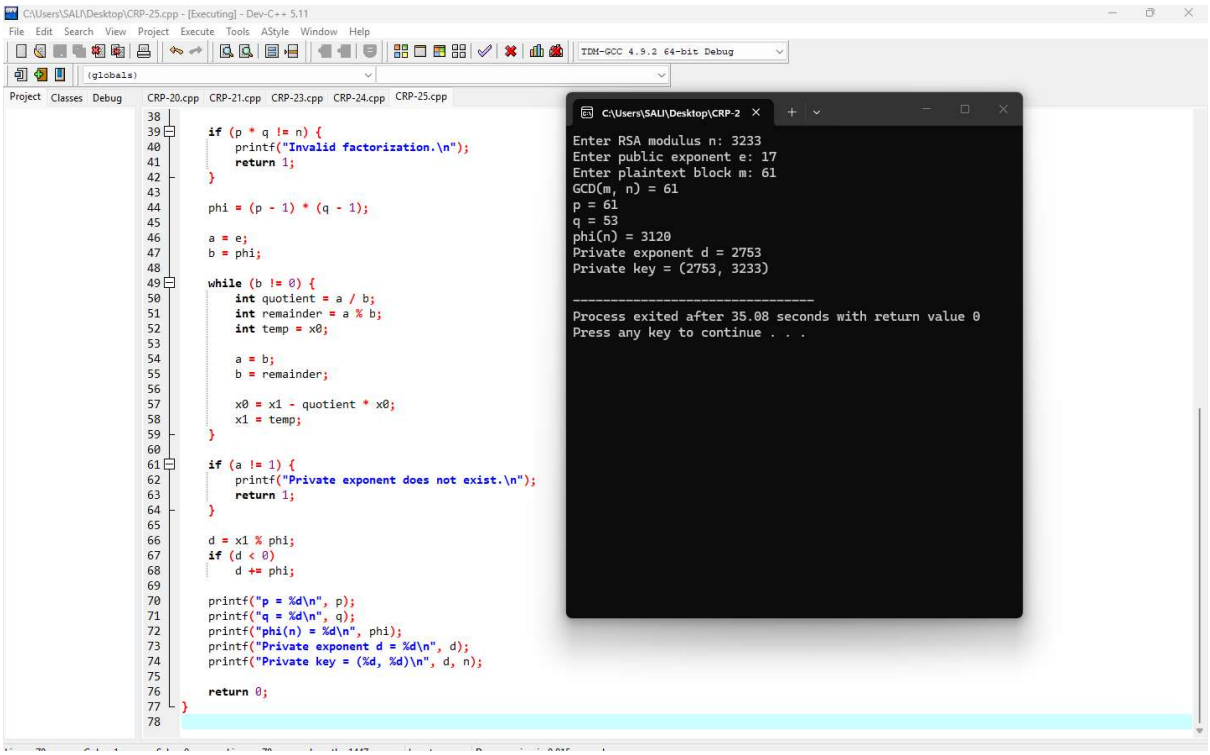
    return 0;
}
```

SAMPLE INPUT

```
Enter RSA modulus n: 3233
Enter public exponent e: 17
Enter plaintext block m: 61
```

SAMPLE OUTPUT

```
GCD(m, n) = 61
p = 61
q = 53
phi(n) = 3120
Private exponent d = 2753
Private key = (2753, 3233)
```



RESULT

Yes, it helps. If a plaintext block has a non-trivial common factor with \(n\), calculating the GCD can reveal a factor of \(n\). This allows the attacker to calculate \(\phi(n)\) and derive the RSA private key, provided the public exponent is valid.

Note: This is a small educational example. Real RSA uses much larger moduli and secure encryption padding.